

CMU 14-455 (2019) · 课程资料包 @ShowMeAI



视频

中英双语字幕



课件

一键打包下载



笔记

官方笔记翻译



代码

作业项目解析



视频 · B 站 [扫码或点击链接]

<https://www.bilibili.com/video/BV1qf4y1J7mX>



课件 & 代码 · 博客 [扫码或点击链接]

<http://blog.showmeai.tech/cmu-14-455/>

高级 SQL

数据库

聚合

内存管理

树索引

排序

查询规划

索引构建

分组

并发控制

查询执行

Awesome AI Courses Notes Cheatsheets 是 [ShowMeAI](#) 资料库的分支系列，覆盖最具知名度的 **TOP50+** 门 AI 课程，旨在为读者和学习者提供一整套高品质中文学习笔记和速查表。

点击课程名称，跳转至课程**资料包**页面，**一键下载**课程全部资料！

机器学习	深度学习	自然语言处理	计算机视觉
Stanford · CS229	Stanford · CS230	Stanford · CS224n	Stanford · CS231n
# Awesome AI Courses Notes Cheatsheets · 持续更新中			
知识图谱	图机器学习	深度强化学习	自动驾驶
Stanford · CS520	Stanford · CS224W	UCBerkeley · CS285	MIT · 6.S094



微信公众号

资料下载方式 2：扫码点击**底部菜单栏**

称为 **AI 内容创作者**？回复 [添砖加瓦]

07 | Tree Indexes –Part I



Intro to Database Systems
15-445/15-645
Fall 2019

AP

Andy Pavlo
Computer Science
Carnegie Mellon University

ADMINISTRIVIA

Project #1 is due Fri Sept 27th @ 11:59pm

Homework #2 is due Mon Sept 30th @ 11:59pm

DATA STRUCTURES

Internal Meta-data

Core Data Storage

Temporary Data Structures

Table Indexes



TABLE INDEXES

A **table index** is a replica of a subset of a table's attributes that are organized and/or sorted for efficient access using a subset of those attributes.

The DBMS ensures that the contents of the table and the index are logically in sync.

TABLE INDEXES

It is the DBMS's job to figure out the best index(es) to use to execute each query.

There is a trade-off on the number of indexes to create per database.

- Storage Overhead
- Maintenance Overhead



TODAY'S AGENDA

B+Tree Overview

Design Decisions

Optimizations



B-TREE FAMILY

There is a specific data structure called a **B-Tree**.

People also use the term to generally refer to a class of balanced tree data structures:

- **B-Tree** (1971)
- **B+Tree** (1973)
- **B*Tree** (1977?)
- **B^{link}-Tree** (1981)



B-TREE FAMILY

There is a specific data structure called

People also use the term to generalize a class of balanced tree data structures

→ **B-Tree** (1971)

→ **B+Tree** (1973)

→ **B*Tree** (1977?)

→ **B^{link}-Tree** (1981)

Efficient Locking for Concurrent Operations on B-Trees

PHILIP L. LEHMAN
Carnegie-Mellon University
and

S. BING YAO
Purdue University

The B-tree and its variants have been found to be highly useful (both theoretically and in practice) for storing large amounts of information, especially on secondary storage devices. We examine the problem of overcoming the inherent difficulty of concurrent operations on such structures, using a practical storage model. A single additional "link" pointer in each node allows a process to easily recover from tree modifications performed by other concurrent processes. Our solution compares favorably with earlier solutions in that the locking scheme is simpler (no read-locks are used) and only a (small) constant number of nodes are locked by any update process at any given time. An informal correctness proof for our system is given.

Key Words and Phrases: database, data structures, B-tree, index organizations, concurrent algorithms, concurrency controls, locking protocols, correctness, consistency, multiway search trees

CR Categories: 3.73, 3.74, 4.32, 4.33, 4.34, 5.24

1. INTRODUCTION

The B-tree [2] and its variants have been widely used in recent years as a data structure for storing large files of information, especially on secondary storage devices [7]. The guaranteed small (average) search, insertion, and deletion time for these structures makes them quite appealing for database applications.

A topic of current interest in database design is the construction of databases that can be manipulated concurrently and correctly by several processes. In this paper, we consider a simple variant of the B-tree (actually of the B*-tree, proposed by Wedekind [15]) especially well suited for use in a concurrent database system.

Methods for concurrent operations on B*-trees have been discussed by Bayer and Schkolnick [3] and others [6, 12, 13]. The solution given in the current paper

Permission to copy without fee all or part of this material is granted provided that the copies are not made or distributed for direct commercial advantage, the ACM copyright notice and the title of the publication and its date appear, and notice is given that copying is by permission of the Association for Computing Machinery. To copy otherwise, or to republish, requires a fee and/or specific permission.

This research was supported by the National Science Foundation under Grant MCS76-16604. Authors' present addresses: P. L. Lehman, Department of Computer Science, Carnegie-Mellon University, Pittsburgh, PA 15213; S. B. Yao, Department of Computer Science and College of Business and Management, University of Maryland, College Park, MD 20742.

© 1981 ACM 0362-5915/81/1200-0650 \$00.75

ACM Transactions on Database Systems, Vol. 6, No. 4, December 1981, Pages 650-670.

B+TREE

A **B+Tree** is a self-balancing tree data structure that keeps data sorted and allows searches, sequential access, insertions, and deletions in **$O(\log n)$** .

- Generalization of a binary search tree in that a node can have more than two children.
- Optimized for systems that read and write large blocks of data.

The Ubiquitous B-Tree

DOUGLAS COMER

Computer Science Department, Purdue University, West Lafayette, Indiana 47907

B-trees have become, de facto, a standard for file organization. File indexes of users, dedicated database systems, and general-purpose access methods have all been proposed and implemented using B-trees. This paper reviews B-trees and shows why they have been so successful. It discusses the major variations of the B-tree, especially the B⁺-tree, contrasting the relative merits and costs of each implementation. It illustrates a general purpose access method which uses a B-tree.

Keywords and Phrases: B-tree, B⁺-tree, B^{*}-tree, file organization, index

CR Categories: 3.73 3.74 4.33 4.34

INTRODUCTION

The secondary storage facilities available on large computer systems allow users to store, update, and recall data from large collections of information called files. A computer must retrieve an item and place it in main memory before it can be processed. In order to make good use of the computer resources, one must organize files intelligently, making the retrieval process efficient.

The choice of a good file organization depends on the kinds of retrieval to be performed. There are two broad classes of retrieval commands which can be illustrated by the following examples:

Sequential: "From our employee file, prepare a list of all employees' names and addresses," and
Random: "From our employee file, extract the information about employee J. Smith".

We can imagine a filing cabinet with three drawers of folders, one folder for each employee. The drawers might be labeled "A-G," "H-R," and "S-Z," while the folders

might be labeled with the employees' last names. A sequential request requires the searcher to examine the entire file, one folder at a time. On the other hand, a random request implies that the searcher, guided by the labels on the drawers and folders, need only extract one folder.

Associated with a large, randomly accessed file in a computer system is an index which, like the labels on the drawers and folders of the file cabinet, speeds retrieval by directing the searcher to the small part of the file containing the desired item. Figure 1 depicts a file and its index. An index may be physically integrated with the file, like the labels on employee folders, or physically separate, like the labels on the drawers. Usually the index itself is a file. If the index file is large, another index may be built on top of it to speed retrieval further, and so on. The resulting hierarchy is similar to the employee file, where the topmost index consists of labels on drawers, and the next level of index consists of labels on folders.

Natural hierarchies, like the one formed by considering last names as index entries, do not always produce the best performance.

Permission to copy without fee all or part of this material is granted provided that the copies are not made or distributed for direct commercial advantage, the ACM copyright notice and the title of the publication and its date appear, and notice is given that copying is by permission of the Association for Computing Machinery. To copy otherwise, or to republish, requires a fee and/or specific permission.

© 1979 ACM 0010-4882/79/0000-0121 \$00.75

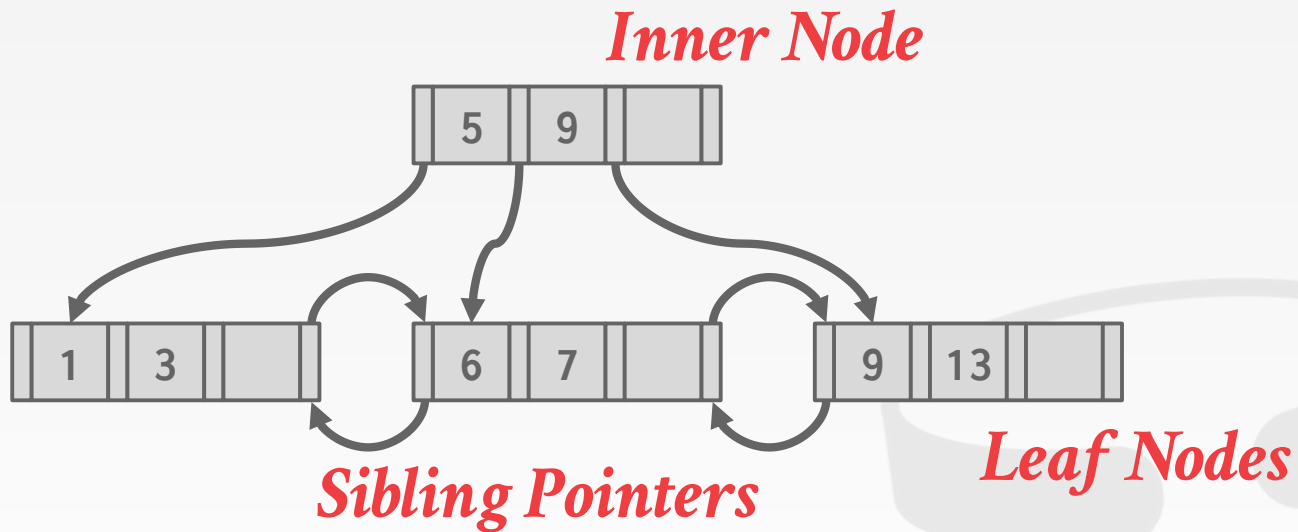
Computing Surveys, Vol 11, No 2, June 1979

B+TREE PROPERTIES

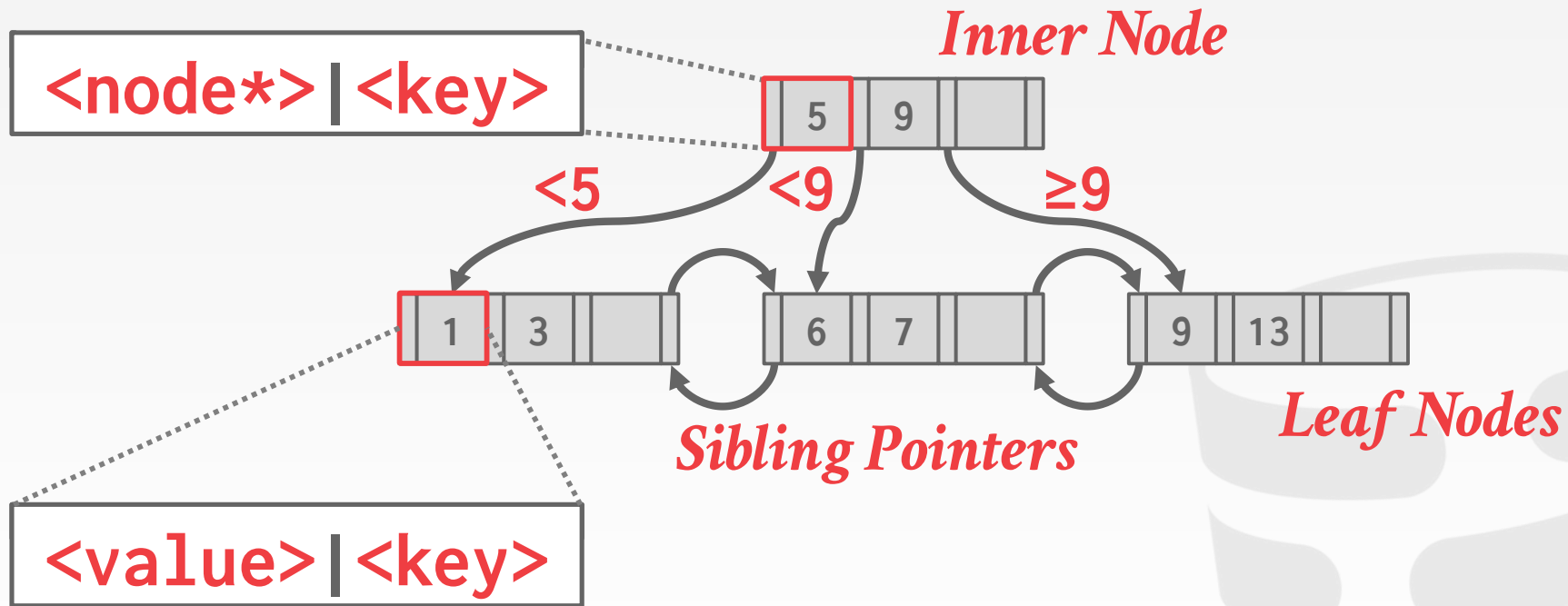
A B+Tree is an **M**-way search tree with the following properties:

- It is perfectly balanced (i.e., every leaf node is at the same depth).
- Every node other than the root, is at least half-full
 $M/2 - 1 \leq \#keys \leq M - 1$
- Every inner node with **k** keys has **k+1** non-null children

B+TREE EXAMPLE



B+TREE EXAMPLE



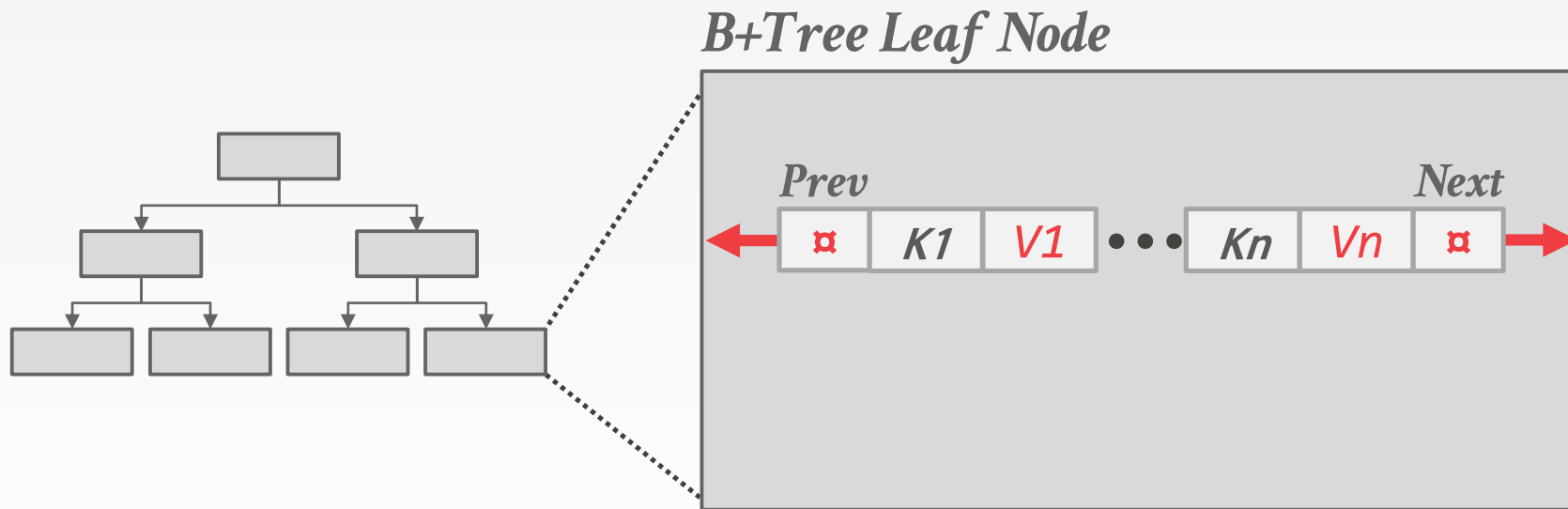
NODES

Every B+Tree node is comprised of an array of key/value pairs.

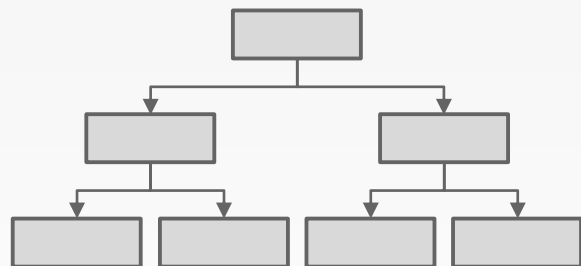
- The keys are derived from the attributes(s) that the index is based on.
- The values will differ based on whether the node is classified as **inner nodes** or **leaf nodes**.

The arrays are (usually) kept in sorted key order.

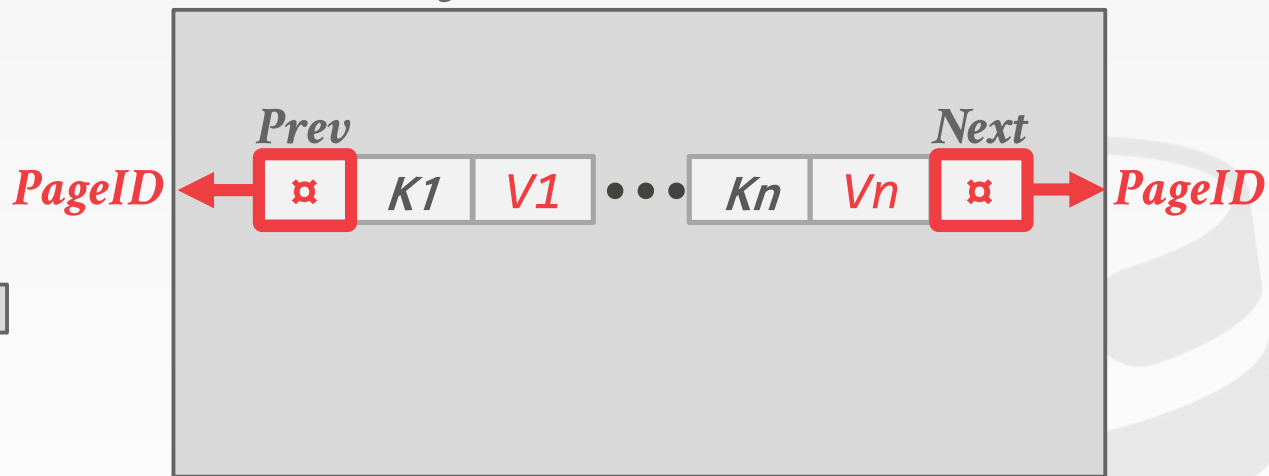
B+TREE LEAF NODES



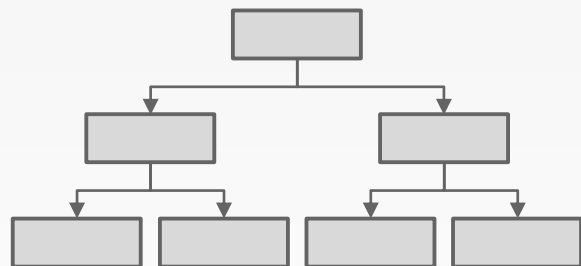
B+TREE LEAF NODES



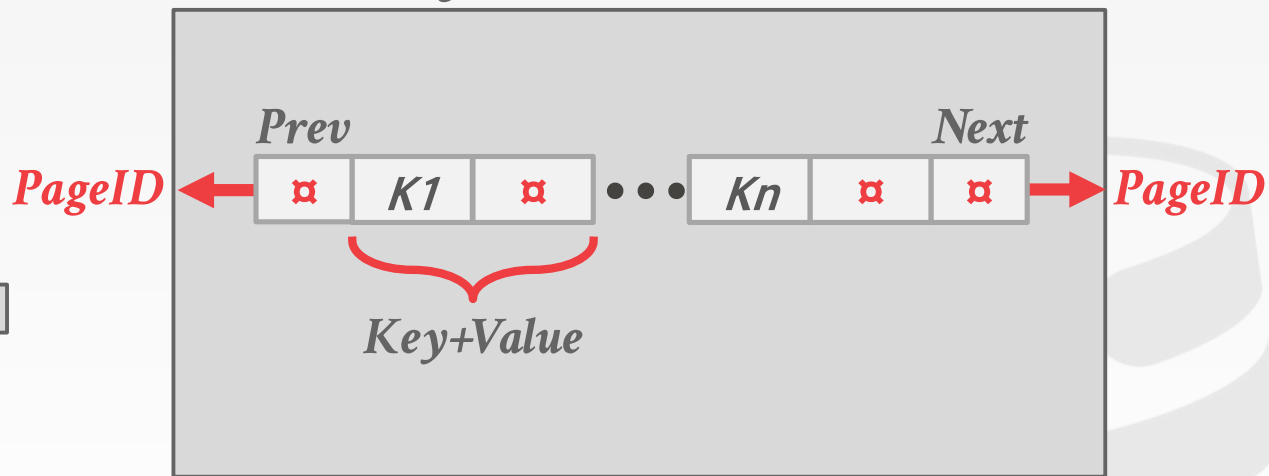
B+Tree Leaf Node



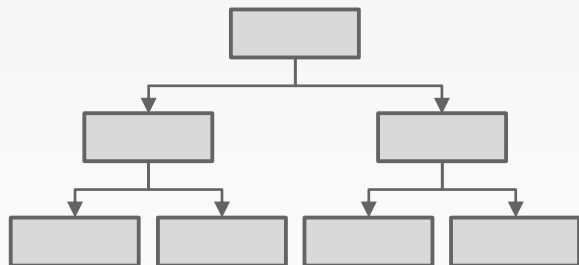
B+TREE LEAF NODES



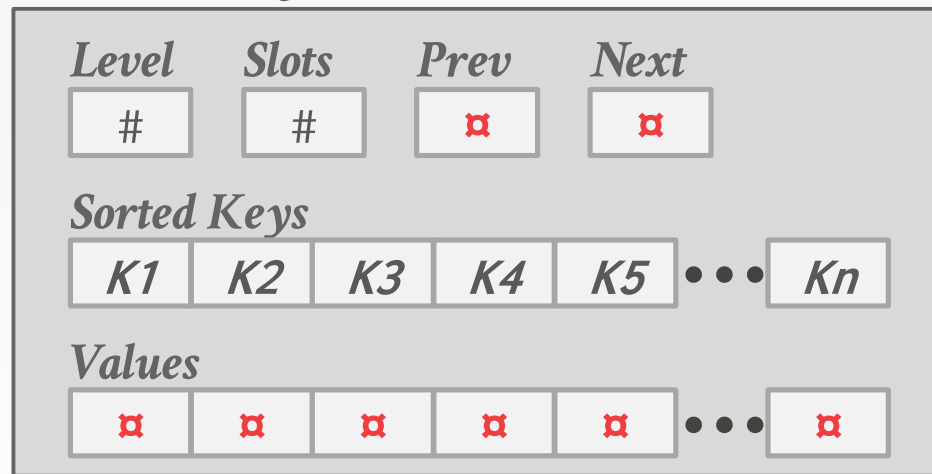
B+Tree Leaf Node



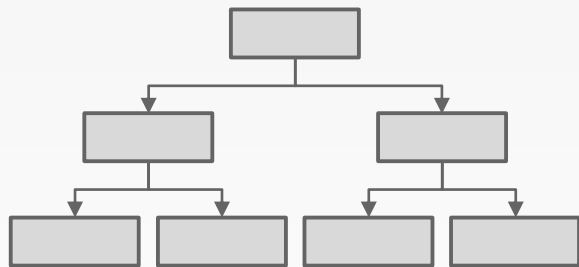
B+TREE LEAF NODES



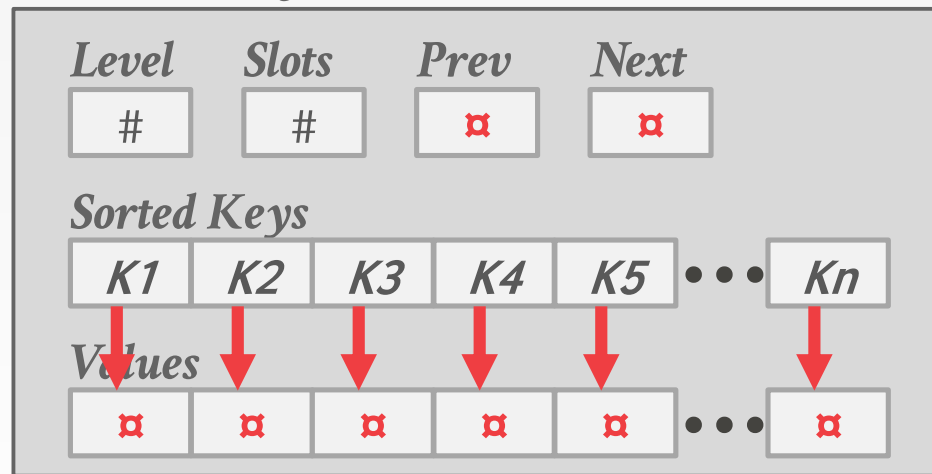
B+Tree Leaf Node



B+TREE LEAF NODES



B+Tree Leaf Node



LEAF NODE VALUES

Approach #1: Record Ids

- A pointer to the location of the tuple that the index entry corresponds to.



PostgreSQL



Microsoft SQL Server



DB2

ORACLE

Approach #2: Tuple Data

- The actual contents of the tuple is stored in the leaf node.
- Secondary indexes have to store the record id as their values.



SQLite



Microsoft SQL Server



MySQL

ORACLE

B-TREE VS. B+TREE

The original **B-Tree** from 1972 stored keys + values in all nodes in the tree.

→ More space efficient since each key only appears once in the tree.

A **B+Tree** only stores values in leaf nodes. Inner nodes only guide the search process.

B+TREE INSERT

Find correct leaf node **L**.

Put data entry into **L** in sorted order.

If **L** has enough space, done!

Otherwise, split **L** keys into **L** and a new node **L2**

→ Redistribute entries evenly, copy up middle key.

→ Insert index entry pointing to **L2** into parent of **L**.

To split inner node, redistribute entries evenly,
but push up middle key.

B+TREE VISUALIZATION

<https://cmudb.io/btree>

Source: David Gales (Univ. of San Francisco)

B+TREE DELETE

Start at root, find leaf **L** where entry belongs.

Remove the entry.

If **L** is at least half-full, done!

If **L** has only **$M/2-1$** entries,

→ Try to re-distribute, borrowing from sibling (adjacent node with same parent as **L**).

→ If re-distribution fails, merge **L** and sibling.

If merge occurred, must delete entry (pointing to **L** or sibling) from parent of **L**.

B+TREES IN PRACTICE

Typical Fill-Factor: 67%.

Typical Capacities:

- Height 4: 1334 = 312,900,721 entries
- Height 3: 1333 = 2,406,104 entries

Pages per level:

- Level 1 = 1 page = 8 KB
- Level 2 = 134 pages = 1 MB
- Level 3 = 17,956 pages = 140 MB



CLUSTERED INDEXES

The table is stored in the sort order specified by the primary key.

→ Can be either heap- or index-organized storage.

Some DBMSs always use a clustered index.

→ If a table doesn't contain a primary key, the DBMS will automatically make a hidden row id primary key.

Other DBMSs cannot use them at all.

SELECTION CONDITIONS

The DBMS can use a B+Tree index if the query provides any of the attributes of the search key.

Example: Index on $\langle a, b, c \rangle$

→ Supported: $(a=5 \text{ AND } b=3)$

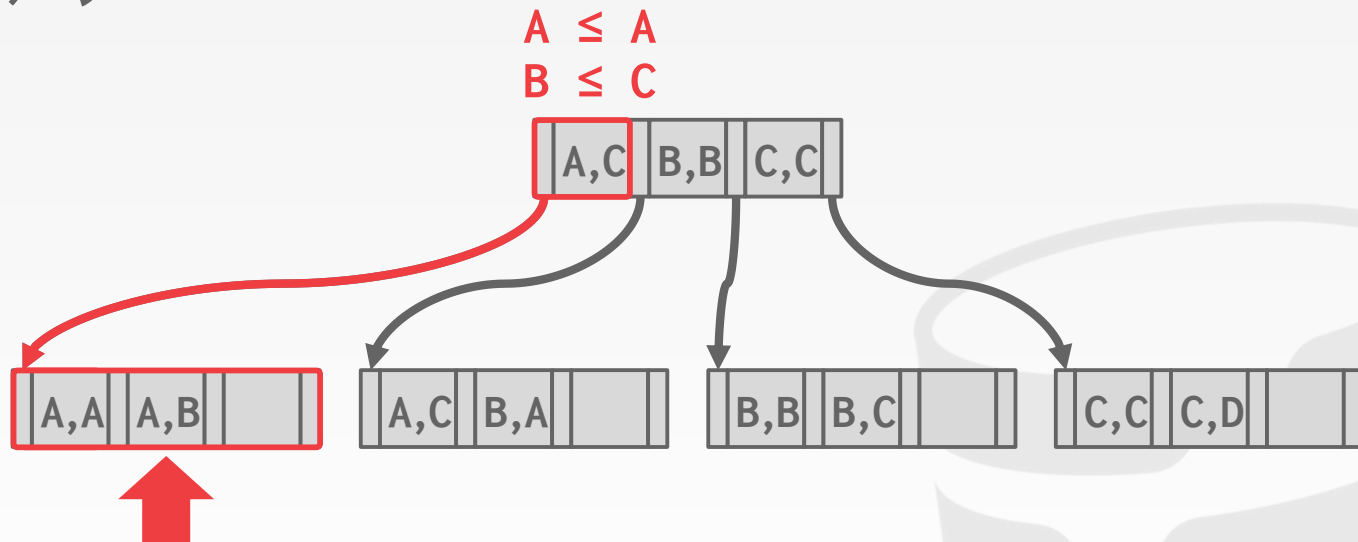
→ Supported: $(b=3)$.

Not all DBMSs support this.

For hash index, we must have all attributes in search key.

SELECTION CONDITIONS

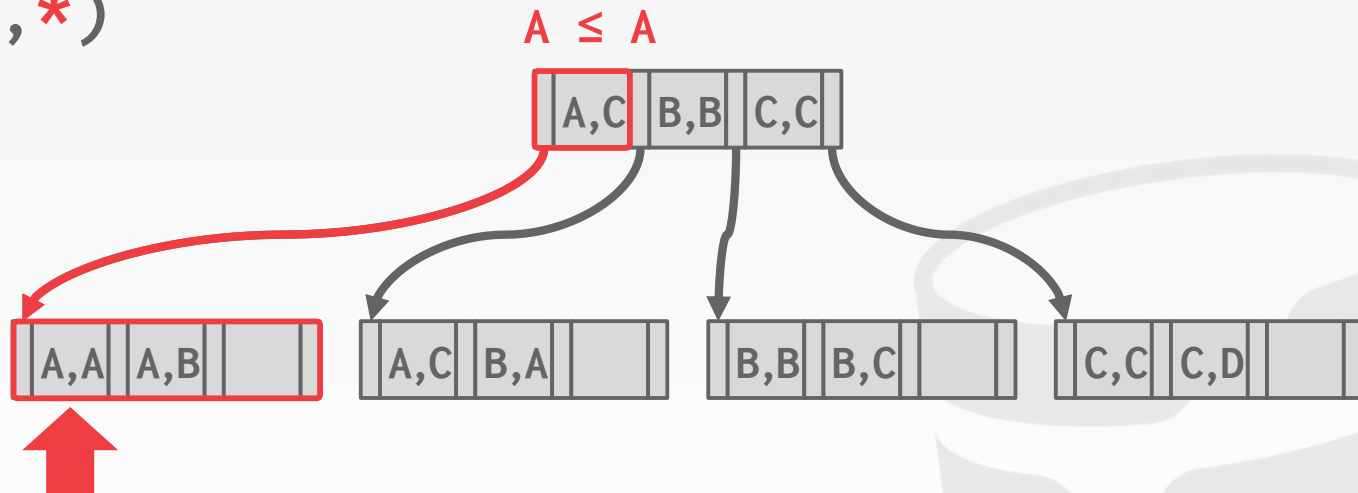
Find Key=(A,B)



SELECTION CONDITIONS

Find Key=(A,B)

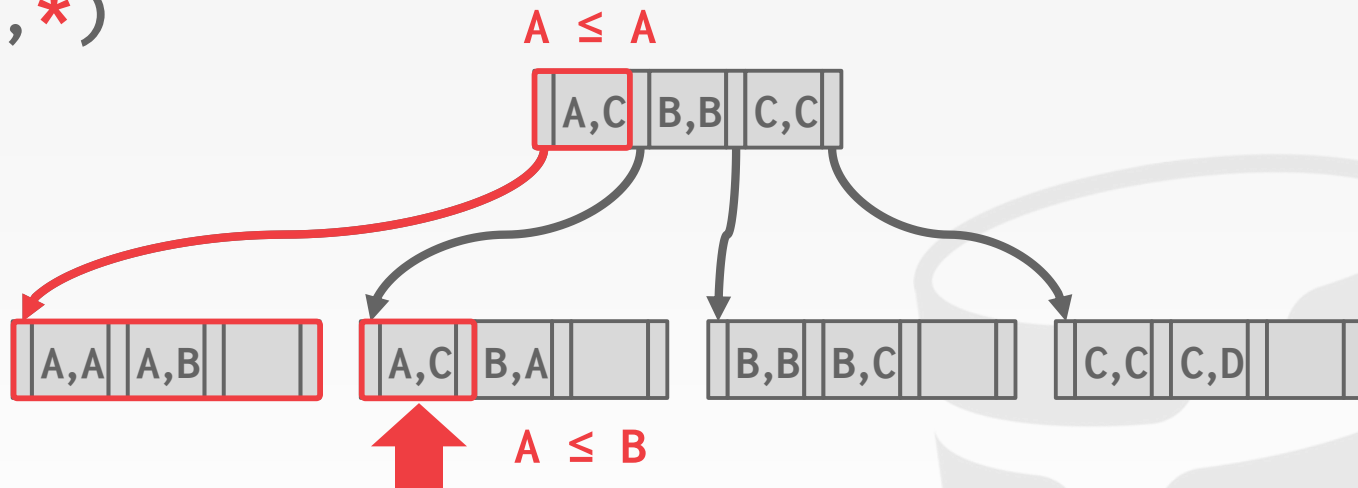
Find Key=(A,*)



SELECTION CONDITIONS

Find Key=(A,B)

Find Key=(A,*)

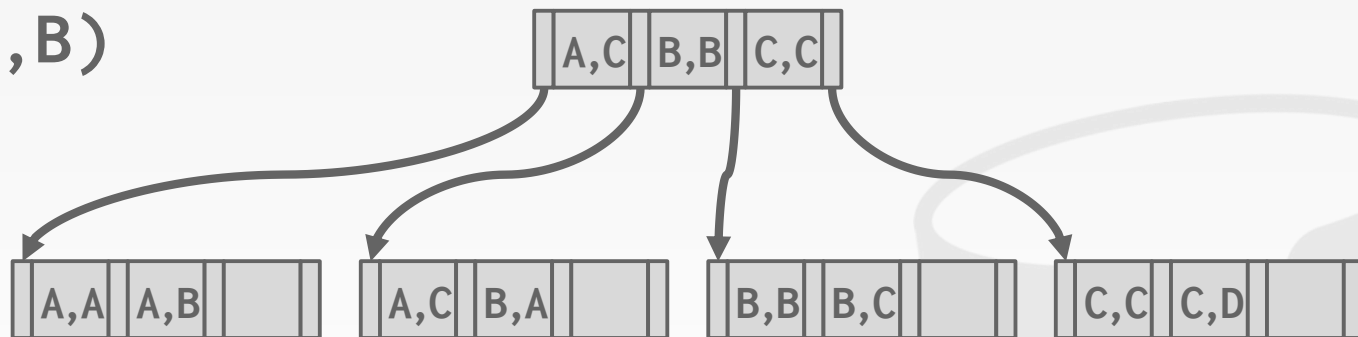


SELECTION CONDITIONS

Find Key=(A,B)

Find Key=(A,*)

Find Key=(*,B)

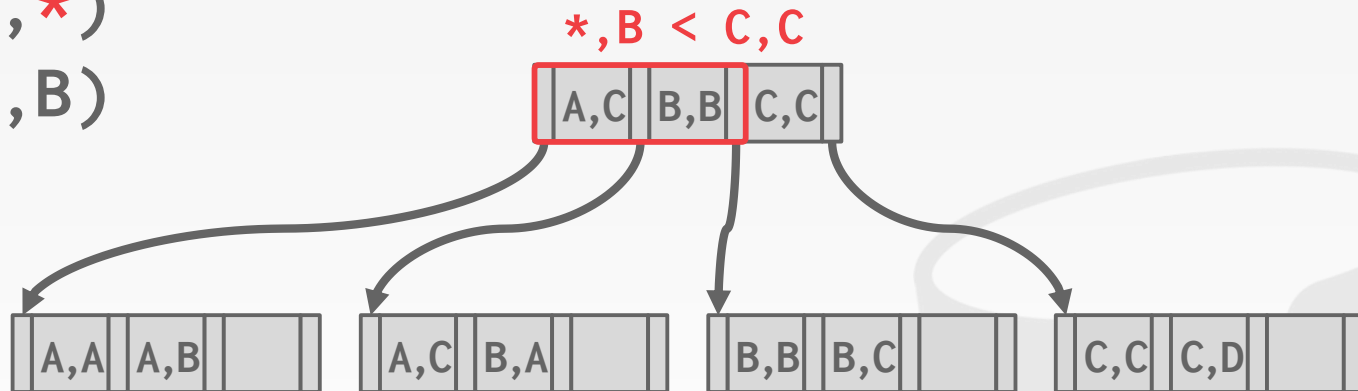


SELECTION CONDITIONS

Find Key=(A,B)

Find Key=(A,*)

Find Key=(*,B)

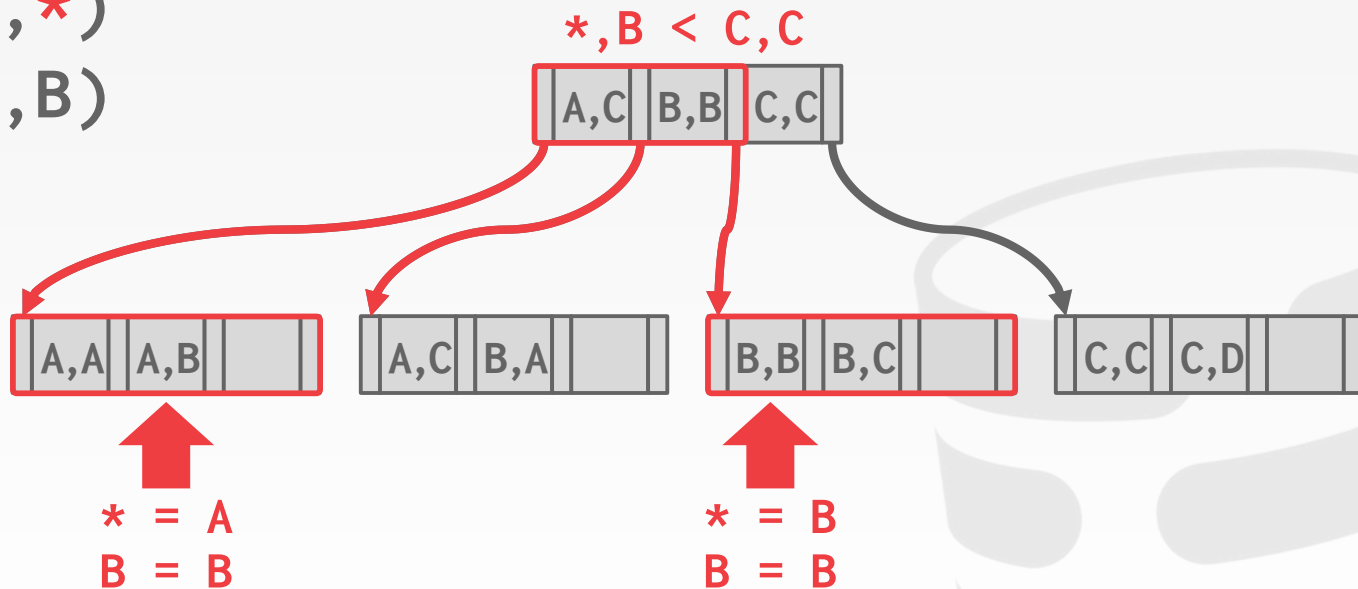


SELECTION CONDITIONS

Find Key=(A,B)

Find Key=(A,*)

Find Key=(*,B)



B+TREE DESIGN CHOICES

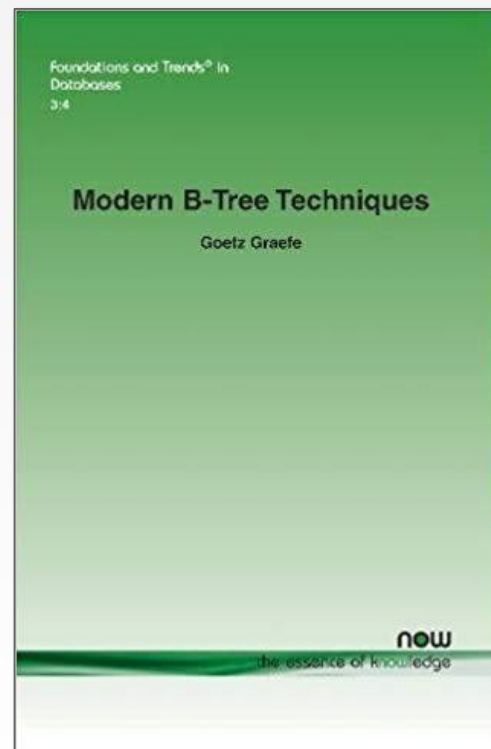
Node Size

Merge Threshold

Variable Length Keys

Non-Unique Indexes

Intra-Node Search



NODE SIZE

The slower the storage device, the larger the optimal node size for a B+Tree.

- HDD ~1MB
- SSD: ~10KB
- In-Memory: ~512B

Optimal sizes can vary depending on the workload

- Leaf Node Scans vs. Root-to-Leaf Traversals

MERGE THRESHOLD

Some DBMSs do not always merge nodes when it is half full.

Delaying a merge operation may reduce the amount of reorganization.

It may also be better to just let underflows to exist and then periodically rebuild entire tree.

VARIABLE LENGTH KEYS

Approach #1: Pointers

→ Store the keys as pointers to the tuple's attribute.

Approach #2: Variable Length Nodes

- The size of each node in the index can vary.
- Requires careful memory management.

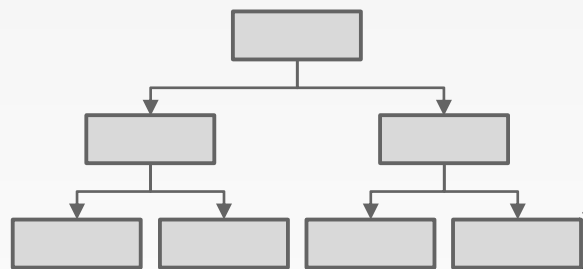
Approach #3: Padding

→ Always pad the key to be max length of the key type.

Approach #4: Key Map / Indirection

→ Embed an array of pointers that map to the key + value list within the node.

KEY MAP / INDIRECTION



B+Tree Leaf Node

<i>Level</i>	<i>Slots</i>	<i>Prev</i>	<i>Next</i>
#	#	❑	❑

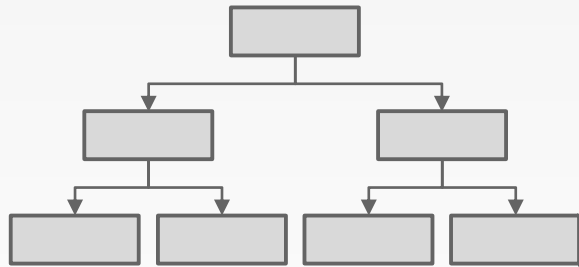
Sorted Key Map

❑	❑	❑	❑
---	---	---	---

Key+Values

		←		Prashanth	V4
Lin	V2	Andy	V1	Obama	V3

KEY MAP / INDIRECTION



B+Tree Leaf Node

Level	Slots	Prev	Next
#	#	❑	❑

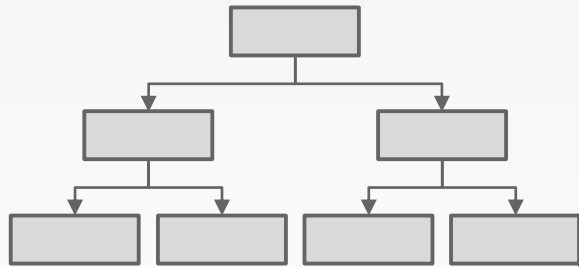
Sorted Key Map

❑	❑	❑	❑
---	---	---	---

Key+Values

Lin	V2	Andy	V1	Obama	V3
				Prashanth	V4

KEY MAP / INDIRECTION



B+Tree Leaf Node

Level	Slots	Prev	Next
#	#	⌘	⌘

Sorted Key Map

A ⌘	L ⌘	O ⌘	P ⌘
-----	-----	-----	-----

Key+Values

				Prashanth	V4
Lin	V2	Andy	V1	Obama	V3

NON-UNIQUE INDEXES

Approach #1: Duplicate Keys

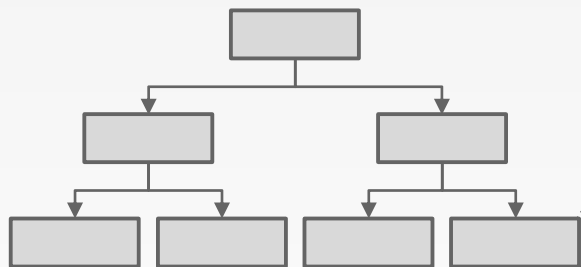
- Use the same leaf node layout but store duplicate keys multiple times.

Approach #2: Value Lists

- Store each key only once and maintain a linked list of unique values.

NON-UNIQUE: DUPLICATE KEYS

B+Tree Leaf Node



Level

#

Slots

#

Prev

❑

Next

❑

Sorted Keys

K1

K1

K1

K2

K2

...

Kn

Values

❑

❑

❑

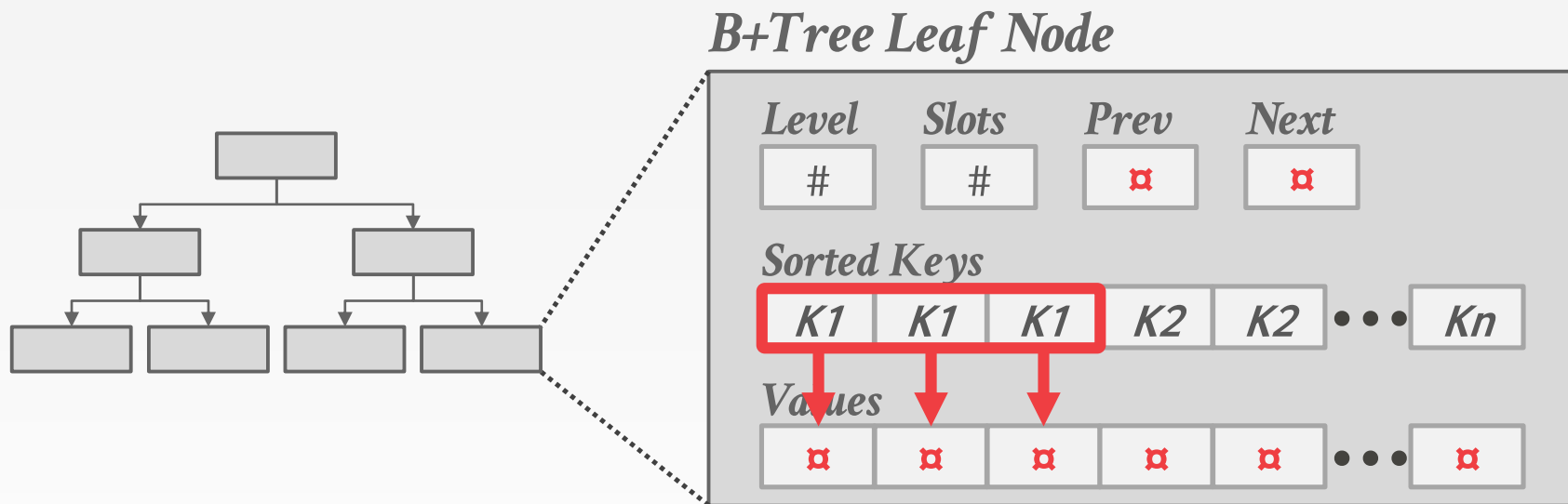
❑

❑

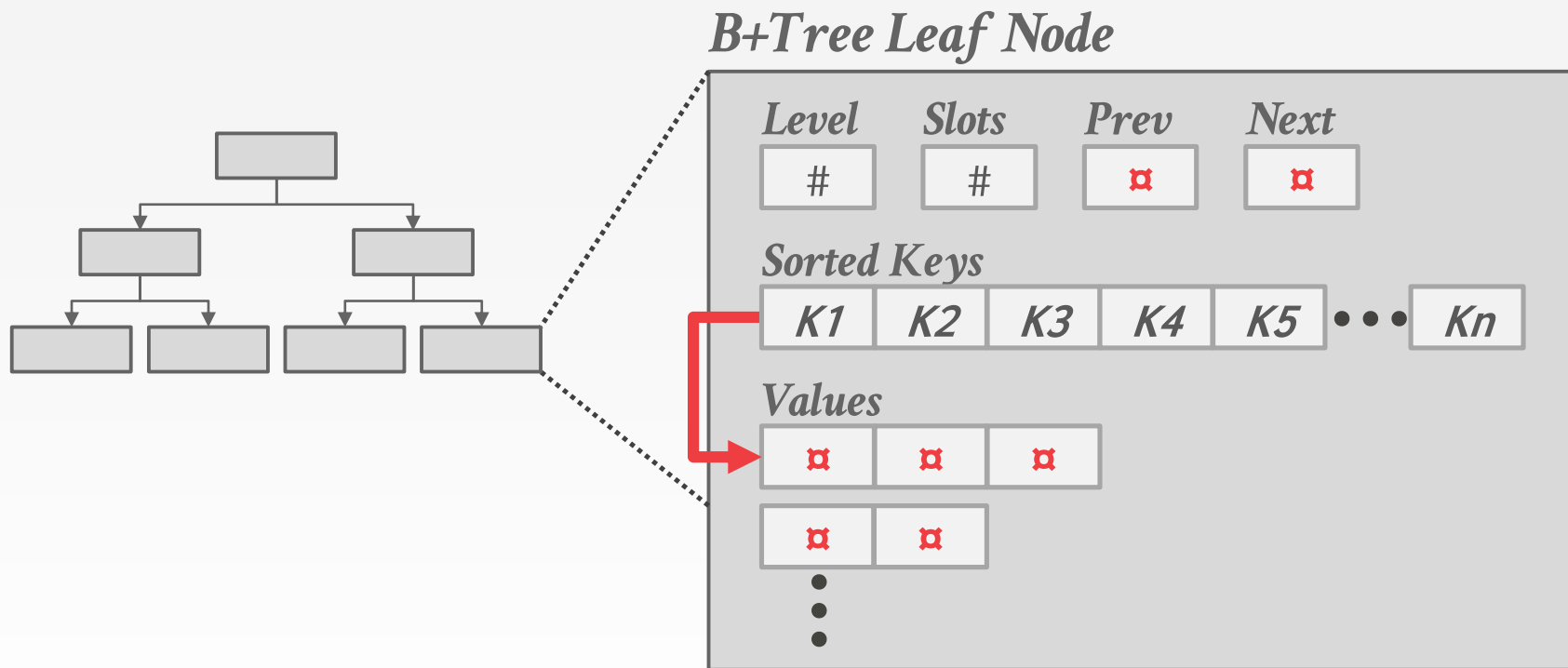
...

❑

NON-UNIQUE: DUPLICATE KEYS



NON-UNIQUE: VALUE LISTS



INTRA-NODE SEARCH

Approach #1: Linear

→ Scan node keys from beginning to end.

Approach #2: Binary

→ Jump to middle key, pivot left/right depending on comparison.

Approach #3: Interpolation

→ Approximate location of desired key based on known distribution of keys.



INTRA-NODE SEARCH

Approach #1: Linear

→ Scan node keys from beginning to end.

Approach #2: Binary

→ Jump to middle key, pivot left/right depending on comparison.

Approach #3: Interpolation

→ Approximate location of desired key based on known distribution of keys.

Find Key=8

4	5	6	7	8	9	10
---	---	---	---	---	---	----



4	5	6	7	8	9	10
---	---	---	---	---	---	----

INTRA-NODE SEARCH

Approach #1: Linear

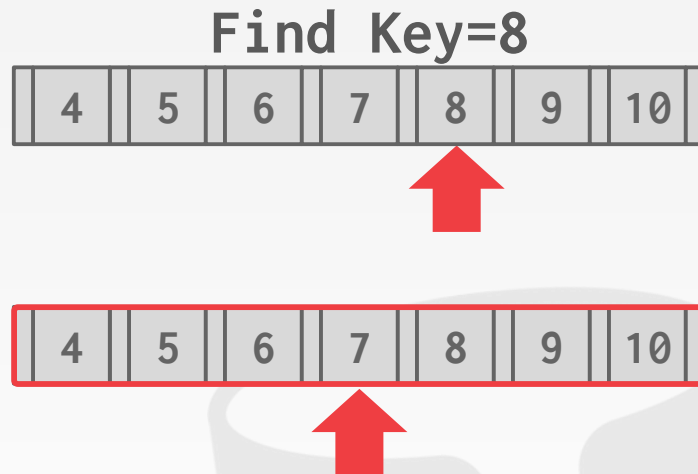
→ Scan node keys from beginning to end.

Approach #2: Binary

→ Jump to middle key, pivot left/right depending on comparison.

Approach #3: Interpolation

→ Approximate location of desired key based on known distribution of keys.



INTRA-NODE SEARCH

Approach #1: Linear

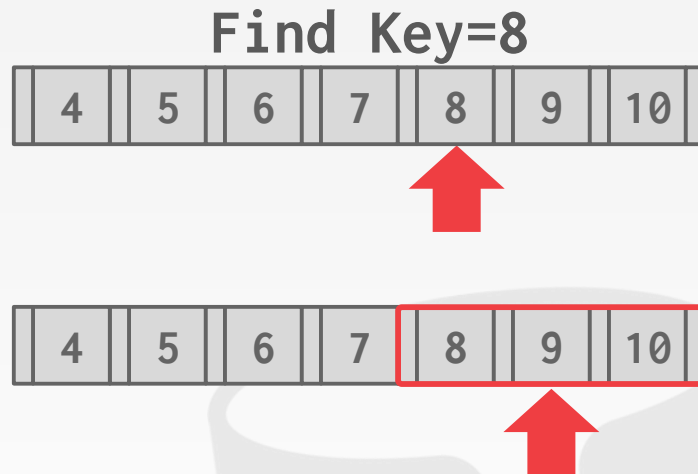
→ Scan node keys from beginning to end.

Approach #2: Binary

→ Jump to middle key, pivot left/right depending on comparison.

Approach #3: Interpolation

→ Approximate location of desired key based on known distribution of keys.



INTRA-NODE SEARCH

Approach #1: Linear

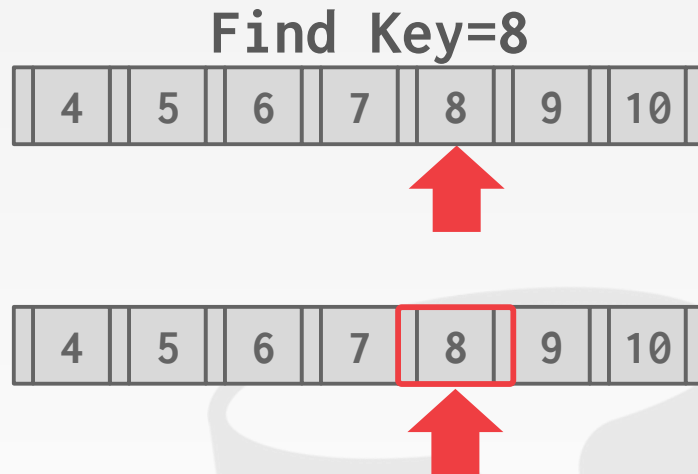
→ Scan node keys from beginning to end.

Approach #2: Binary

→ Jump to middle key, pivot left/right depending on comparison.

Approach #3: Interpolation

→ Approximate location of desired key based on known distribution of keys.



INTRA-NODE SEARCH

Approach #1: Linear

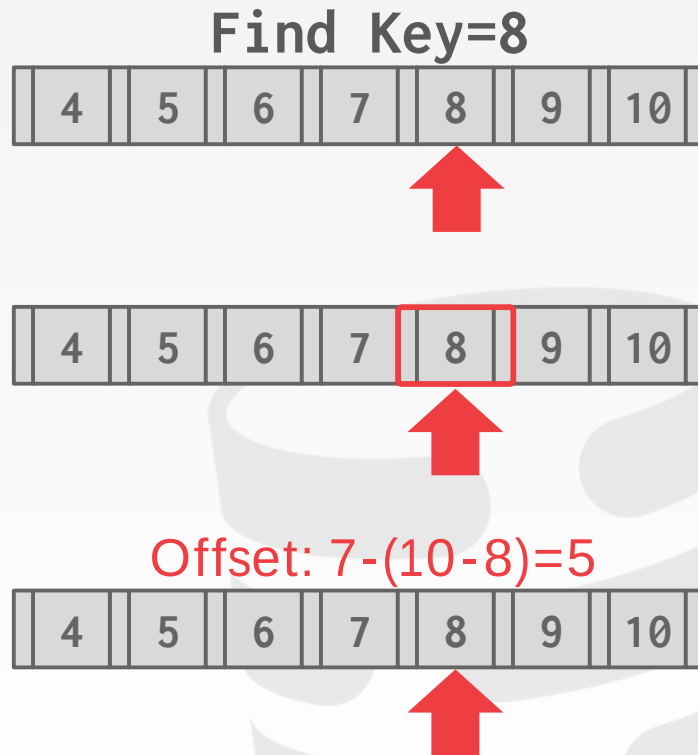
→ Scan node keys from beginning to end.

Approach #2: Binary

→ Jump to middle key, pivot left/right depending on comparison.

Approach #3: Interpolation

→ Approximate location of desired key based on known distribution of keys.



OPTIMIZATIONS

Prefix Compression

Suffix Truncation

Bulk Insert

Pointer Swizzling



PREFIX COMPRESSION

Sorted keys in the same leaf node are likely to have the same prefix.

Instead of storing the entire key each time, extract common prefix and store only unique suffix for each key.

→ Many variations.

robbed	robbing	robot
--------	---------	-------



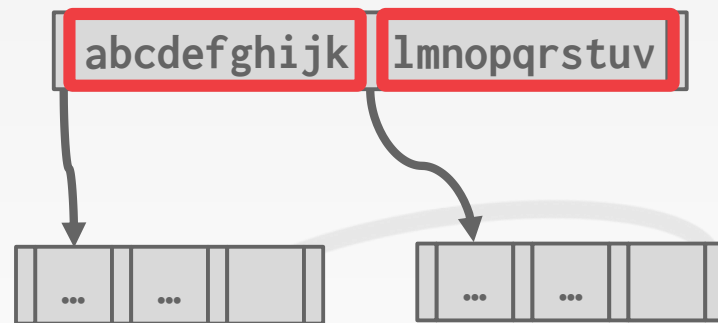
<i>Prefix: rob</i>		
bed	bing	ot

SUFFIX TRUNCATION

The keys in the inner nodes are only used to "direct traffic".

→ We don't need the entire key.

Store a minimum prefix that is needed to correctly route probes into the index.

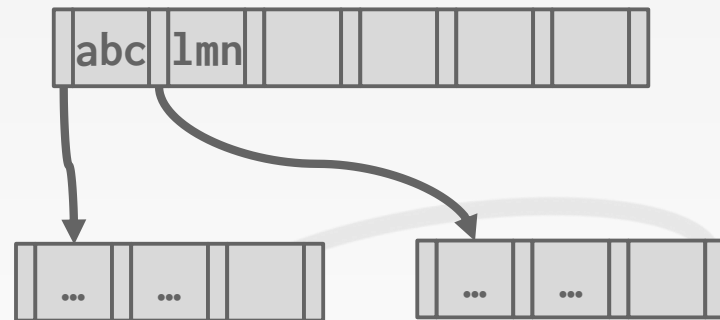


SUFFIX TRUNCATION

The keys in the inner nodes are only used to "direct traffic".

→ We don't need the entire key.

Store a minimum prefix that is needed to correctly route probes into the index.

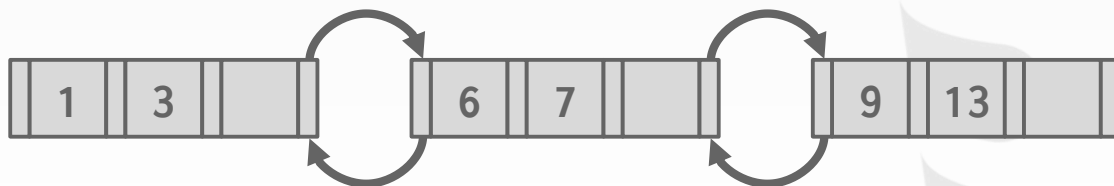


BULK INSERT

The fastest/best way to build a B+Tree is to first sort the keys and then build the index from the bottom up.

Keys: 3, 7, 9, 13, 6, 1

Sorted Keys: 1, 3, 6, 7, 9, 13

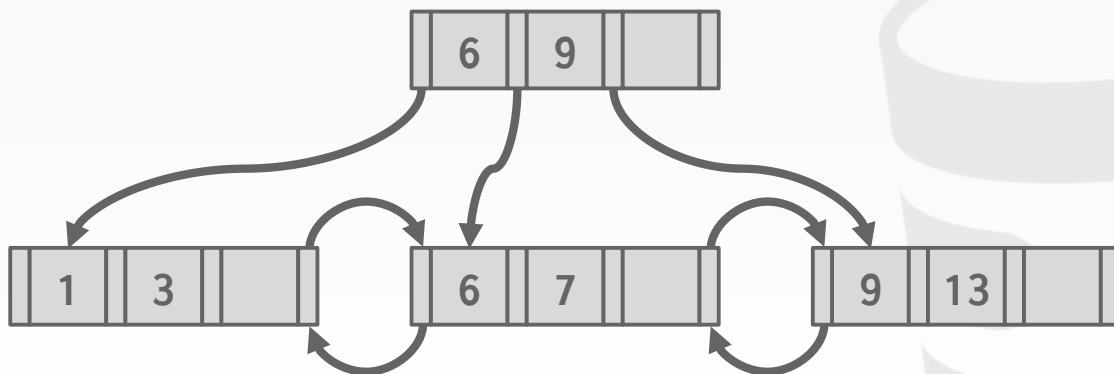


BULK INSERT

The fastest/best way to build a B+Tree is to first sort the keys and then build the index from the bottom up.

Keys: 3, 7, 9, 13, 6, 1

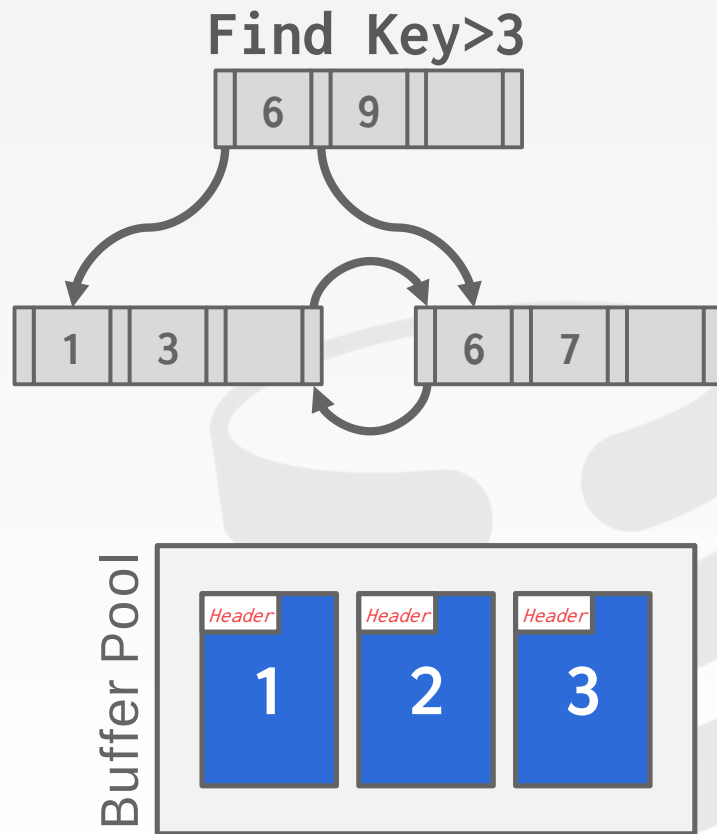
Sorted Keys: 1, 3, 6, 7, 9, 13



POINTER SWIZZLING

Nodes use page ids to reference other nodes in the index. The DBMS must get the memory location from the page table during traversal.

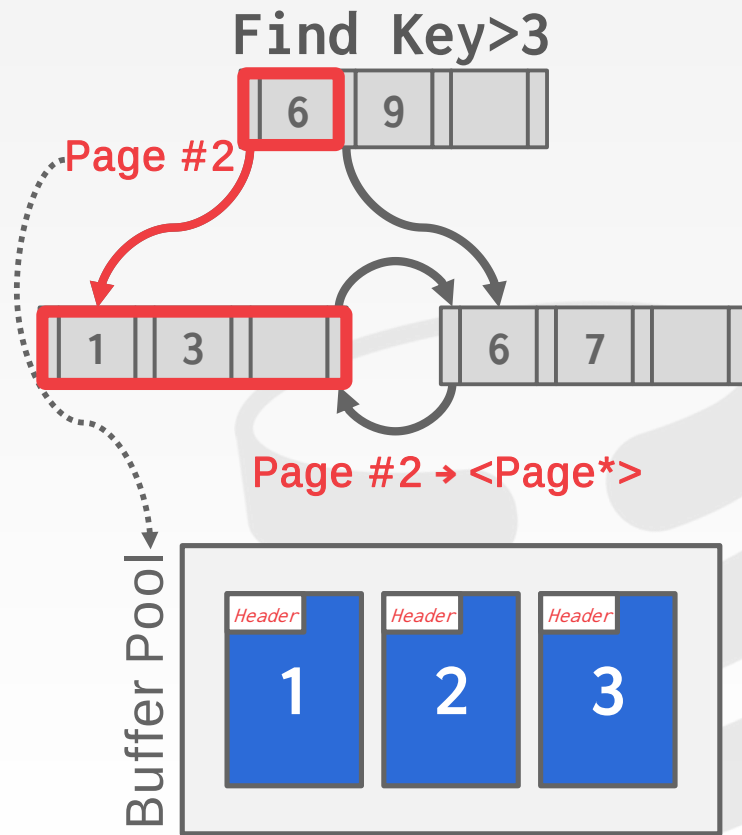
If a page is pinned in the buffer pool, then we can store raw pointers instead of page ids. This avoids address lookups from the page table.



POINTER SWIZZLING

Nodes use page ids to reference other nodes in the index. The DBMS must get the memory location from the page table during traversal.

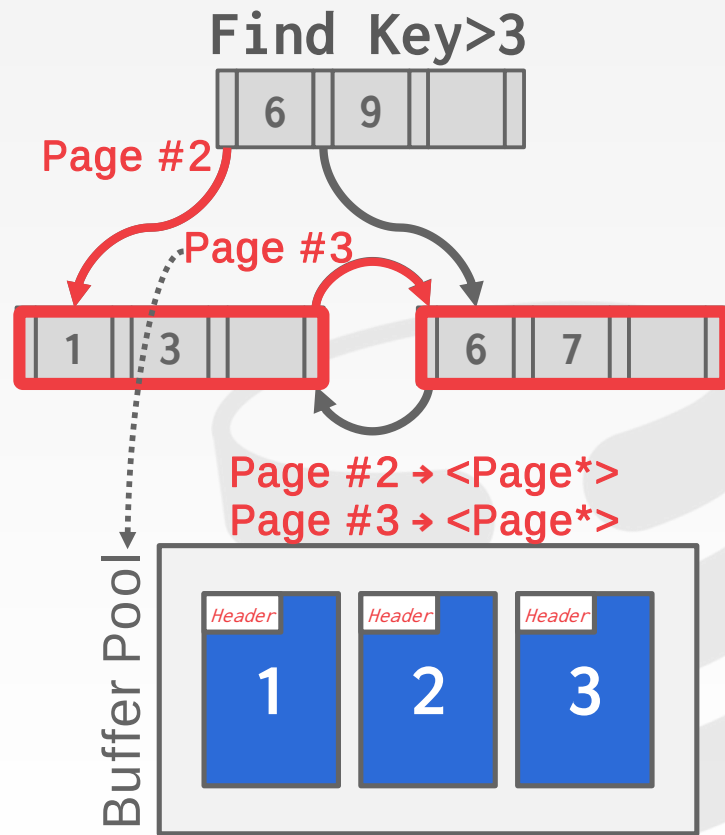
If a page is pinned in the buffer pool, then we can store raw pointers instead of page ids. This avoids address lookups from the page table.



POINTER SWIZZLING

Nodes use page ids to reference other nodes in the index. The DBMS must get the memory location from the page table during traversal.

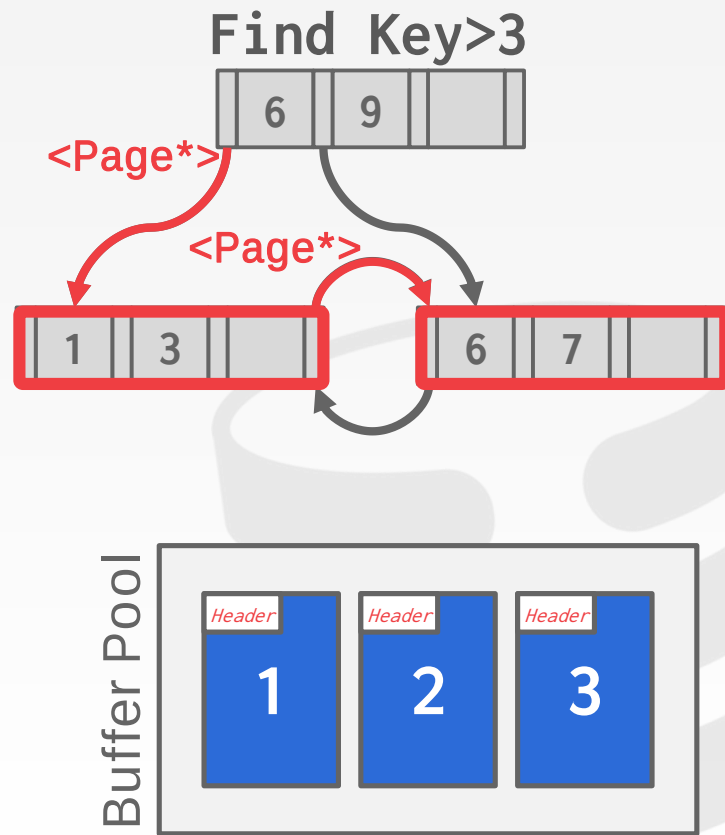
If a page is pinned in the buffer pool, then we can store raw pointers instead of page ids. This avoids address lookups from the page table.



POINTER SWIZZLING

Nodes use page ids to reference other nodes in the index. The DBMS must get the memory location from the page table during traversal.

If a page is pinned in the buffer pool, then we can store raw pointers instead of page ids. This avoids address lookups from the page table.



CONCLUSION

The venerable B+Tree is always a good choice for your DBMS.

NEXT CLASS

More B+Trees

Tries / Radix Trees

Inverted Indexes



CMU 14-455 (2019) · 课程资料包 @ShowMeAI



视频

中英双语字幕



课件

一键打包下载



笔记

官方笔记翻译



代码

作业项目解析



视频 · B 站 [扫码或点击链接]

<https://www.bilibili.com/video/BV1qf4y1J7mX>



课件 & 代码 · 博客 [扫码或点击链接]

<http://blog.showmeai.tech/cmu-14-455/>

高级 SQL
数据库

聚合
内存管理
树索引

排序
查询规划
索引构建

并发控制
查询执行

Awesome AI Courses Notes Cheatsheets 是 [ShowMeAI](#) 资料库的分支系列，覆盖最具知名度的 **TOP50+** 门 AI 课程，旨在为读者和学习者提供一整套高品质中文学习笔记和速查表。

点击课程名称，跳转至课程**资料包**页面，**一键下载**课程全部资料！

机器学习	深度学习	自然语言处理	计算机视觉
Stanford · CS229	Stanford · CS230	Stanford · CS224n	Stanford · CS231n
# Awesome AI Courses Notes Cheatsheets · 持续更新中			
知识图谱	图机器学习	深度强化学习	自动驾驶
Stanford · CS520	Stanford · CS224W	UCBerkeley · CS285	MIT · 6.S094



微信公众号

资料下载方式 2：扫码点击**底部菜单栏**
称为 **AI 内容创作者**？回复 [添砖加瓦]